INTERVIEW PREP SERIES

Express.js

Interview Questions & Easy Answers

---

52 questions · 19 marked ★ most asked

Covers routing, middleware, request/response, REST API design, error handling, authentication with JWT, file uploads, validation, security and project structure for MERN backend interviews.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics

Q1. What is Express.js? ★ Most asked

Express is a minimal and flexible web framework for Node.js. It makes building servers and REST APIs much easier than plain `http` by giving simple routing, middleware support, and helpers for requests and responses. It is the "E" in MERN.

Q2. Why use Express instead of the plain Node http module? ★ Most asked

With plain `http`, you must manually parse URLs, read request bodies, set headers and handle routes using if-else. Express gives clean routing (`app.get`), middleware, JSON parsing, static files, and error handling with very little code.

Q3. How do you create a basic Express server? ★ Most asked

```
const express = require("express");
const app = express();

app.use(express.json()); // parse JSON bodies

app.get("/", (req, res) => {
  res.send("Hello from Express");
});

app.listen(5000, () => console.log("Server running on 5000"));
```

Q4. What is the difference between `app.listen` and `http.createServer(app)`?

`app.listen()` creates the HTTP server for you. You use `http.createServer(app)` when you need direct access to the server object, e.g. to attach Socket.io.

Routing

Q5. What is routing in Express? ★ Most asked

Routing decides how the app responds to a request based on the URL path and HTTP method.

```
app.get("/users", getUsers);
app.post("/users", createUser);
app.put("/users/:id", updateUser);
app.delete("/users/:id", deleteUser);
```

Q6. What is the difference between `req.params`, `req.query` and `req.body`? ★ Most asked

- `req.params`: values from the URL path. `/users/:id` with `/users/42` gives `req.params.id = "42"`.
- `req.query`: values after `?`. `/search?q=phone&page=2` gives `req.query.q = "phone"`.
- `req.body`: data sent in the request body (POST/PUT). It needs `express.json()` middleware.

Q7. What is `express.Router()`? ★ Most asked

Router creates modular, mountable route handlers so you can split routes into separate files.

```
// routes/userRoutes.js
const router = express.Router();
router.get("/", getUsers);
```

```
router.post("/", createUser);
module.exports = router;

// server.js
app.use("/api/users", userRoutes);
```

Q8. What are route parameters and how do you make one optional?

Route parameters are named URL segments like `:id`. In Express 4 you can make one optional with `?`: `/users/:id?`. Express 5 changed the path syntax and uses braces for optional parts: `/users{/:id}`.

Q9. How do you handle 404 routes?

Add a middleware after all routes; it runs only if no route matched.

```
app.use((req, res) => res.status(404).json({ message: "Route not found" }));
```

Q10. What is `app.all()` and `app.route()`?

`app.all("/x", fn)` handles every HTTP method on a path.
`app.route("/users").get(fn).post(fn)` chains handlers for one path, avoiding repeated paths.

Middleware

Q11. What is middleware in Express? ★ Most asked

Middleware is a function that has access to `req`, `res` and `next`. It runs in the middle of the request-response cycle and can change the request, end the response, or call `next()` to pass control to the next middleware.

```
function logger(req, res, next) {
  console.log(req.method, req.url);
  next();
}
app.use(logger);
```

Q12. What are the types of middleware? ★ Most asked

- **Application-level**: `app.use(fn)`.
- **Router-level**: `router.use(fn)`.
- **Built-in**: `express.json()`, `express.urlencoded()`, `express.static()`.
- **Third-party**: `cors`, `morgan`, `helmet`, `cookie-parser`, `multer`.
- **Error-handling**: has four arguments `(err, req, res, next)`.

Q13. What does `next()` do? What happens if you don't call it? ★ Most asked

`next()` passes control to the next middleware or route handler. If you neither call `next()` nor send a response, the request hangs until it times out. `next(err)` skips to the error-handling middleware.

Q14. Does the order of middleware matter?

Yes. Express runs middleware in the order you add them. For example, `express.json()` must come before routes that read `req.body`, and the 404 and error handlers must come last.

Q15. What is `express.static()`?

It serves static files like HTML, CSS, images and the React build folder directly.
`app.use(express.static("public"))` makes `public/logo.png` available at `/logo.png`.

Q16. What is the difference between `express.json()` and `express.urlencoded()`?

`express.json()` parses requests with a JSON body (`Content-Type: application/json`), used by React/fetch. `express.urlencoded({ extended: true })` parses HTML form submissions (`application/x-www-form-urlencoded`).

Q17. What is morgan?

A logging middleware that prints each request's method, URL, status and response time in the console: `app.use(morgan("dev"))`.

Request and response

Q18. What is the difference between `res.send()`, `res.json()` and `res.end()`? ★ Most asked

- `res.send()` sends any type (string, HTML, object) and sets the content type automatically.
- `res.json()` sends JSON and sets `Content-Type: application/json`. Best for APIs.
- `res.end()` ends the response without data (comes from Node).

Q19. How do you set status codes?

`res.status(201).json(newUser)`, `res.status(404).json({ message: "Not found" })`, `res.sendStatus(204)`.

Q20. What is the "Cannot set headers after they are sent" error?

It happens when you try to send a response twice for one request, usually because you forgot to `return` after an early `res.json()`.

```
if (!user) return res.status(404).json({ msg: "No user" });
res.json(user);
```

Q21. How do you redirect in Express?

`res.redirect("/login")` or with a status code `res.redirect(301, "/new-url")`.

Q22. How do you read headers and cookies?

Headers: `req.headers.authorization` or `req.get("Authorization")`. Cookies: use `cookie-parser` then `req.cookies.token`. Set cookies with `res.cookie("token", t, { httpOnly: true })`.

Error handling

Q23. How do you handle errors in Express? ★ Most asked

Create an error-handling middleware with 4 parameters, placed after all routes:

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({ message: err.message || "Server error" });
});
```

In routes, pass errors with `next(err)`.

Q24. How do you handle errors in async route handlers? ★ Most asked

In Express 4, errors thrown in async functions are not caught automatically, so you wrap them with try/catch or a helper.

```
const asyncHandler = fn => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

app.get("/users", asyncHandler(async (req, res) => {
  const users = await User.find();
  res.json(users);
}));
```

Express 5 forwards rejected promises to the error handler automatically.

Q25. What is a custom error class?

A class extending `Error` with a status code, so you can throw meaningful errors: `throw new AppError("User not found", 404)`, and the error middleware reads `err.statusCode`.

REST API design

Q26. How would you design a REST API for a to-do app? ★ Most asked

```
GET    /api/todos        -> list all
GET    /api/todos/:id    -> get one
POST   /api/todos        -> create
PATCH  /api/todos/:id    -> update (e.g. mark done)
DELETE /api/todos/:id    -> delete
```

Use nouns (not verbs) in URLs, correct HTTP methods and status codes, JSON responses, and consistent error formats.

Q27. What is the difference between PUT and PATCH? ★ Most asked

PUT replaces the whole resource with what you send. PATCH updates only the fields you send. Example: to change only the title of a todo, use PATCH.

Q28. What does stateless mean in REST?

Each request contains everything the server needs (like the auth token). The server does not remember previous requests, which makes it easier to scale across many servers.

Q29. What is the MVC pattern in Express? ★ Most asked

- **Model**: data and database schema (Mongoose models).
- **View**: what the user sees (React frontend, or a template engine).
- **Controller**: logic that handles a request, talks to the model and sends the response.

Typical folders: `models/`, `controllers/`, `routes/`, `middleware/`, `config/`.

Q30. How do you implement pagination?

Read `page` and `limit` from the query string and use skip/limit in the database.

```
const page = Number(req.query.page) || 1;
const limit = Number(req.query.limit) || 10;
const items = await Product.find().skip((page - 1) * limit).limit(limit);
```

Q31. How do you implement search and filtering?

Read filters from `req.query` and build the database query: `Product.find({ category: req.query.category, price: { $lte: req.query.max } })`. For text search, use a regex or a text index.

Q32. What is API versioning?

Putting a version in the URL (`/api/v1/users`) or a header, so you can change the API later without breaking old clients.

Authentication and security

Q33. How do you implement JWT authentication in Express? ★ Most asked

1. **Register**: hash the password with bcrypt and save the user.
2. **Login**: find the user, compare the password, and send a JWT.
3. **Protect routes** with middleware:

```
function protect(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ msg: "No token" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ msg: "Invalid token" });
  }
}
app.get("/api/profile", protect, getProfile);
```

Q34. How do you implement role-based access (admin routes)?

After the auth middleware, add a middleware that checks the role.

```
const adminOnly = (req, res, next) =>
  req.user.role === "admin" ? next() : res.status(403).json({ msg: "Forbidden" });
```

Q35. What is the difference between sessions and JWT?

Sessions store login data on the server and give the browser a session ID cookie. JWT stores the data inside a signed token on the client, so the server stays stateless. Sessions are easier to revoke; JWTs scale more easily.

Q36. How do you enable CORS in Express? ★ Most asked

```
const cors = require("cors");
app.use(cors({ origin: "http://localhost:5173", credentials: true }));
```

This adds headers like `Access-Control-Allow-Origin` so the browser allows the frontend to call the API.

Q37. What is helmet?

A middleware that sets secure HTTP headers (like Content-Security-Policy and X-Frame-Options) to protect against common attacks such as clickjacking and XSS.

Q38. How do you add rate limiting?

Use `express-rate-limit` to restrict how many requests one IP can make in a time window, which protects login routes from brute-force attacks.

```
app.use("/api/auth", rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));
```

Q39. How do you validate request data?

Use libraries like **express-validator**, **Joi** or **Zod** to check fields (required, email format, length) before saving, and return a 400 with clear messages on failure.

Files, config and deployment

Q40. How do you upload files in Express? ★ Most asked

Use the **multer** middleware, which handles `multipart/form-data`.

```
const upload = multer({ dest: "uploads/" });
app.post("/upload", upload.single("avatar"), (req, res) => {
  res.json({ file: req.file.filename });
});
```

In production, files are usually uploaded to cloud storage like AWS S3 or Cloudinary.

Q41. How do you connect Express to MongoDB?

```
const mongoose = require("mongoose");
mongoose.connect(process.env.MONGO_URI)
  .then(() => app.listen(5000))
  .catch(err => console.error(err));
```

Q42. What is a template engine?

A tool like EJS, Pug or Handlebars that renders HTML on the server with dynamic data: `res.render("profile", { user })`. In MERN apps, React usually replaces this.

Q43. How do you serve a React build from Express?

```
app.use(express.static(path.join(__dirname, "client/dist")));
app.get("*", (req, res) => res.sendFile(path.join(__dirname,
"client/dist/index.html")));
```

The catch-all route lets React Router handle frontend URLs. In Express 5 the wildcard must be named, e.g. `app.get("/*splat", ...)`.

Q44. How do you test an Express API?

Manually with **Postman** or Thunder Client. Automatically with **Jest** + **Supertest**, which can send requests to your app and check the status and body.

Q45. How would you deploy an Express app?

Push the code to GitHub, set environment variables on the host, and deploy on Render, Railway, or an AWS EC2 instance (with PM2 and Nginx as a reverse proxy). Use MongoDB Atlas for the database.

Q46. What is the difference between Express and NestJS?

Express is minimal and unopinionated; you decide the structure. NestJS is a full framework on top of Express (or Fastify) that uses TypeScript, modules, decorators and dependency injection, making large codebases more organised.

Q47. What is `app.locals` and `res.locals`?

`app.locals` stores values available across the whole app (like the site name). `res.locals` stores values only for the current request, often set by middleware for later handlers to use.

Q48. What is the `trust proxy` setting?

When your app sits behind a proxy like Nginx or a cloud load balancer, `app.set("trust proxy", 1)` tells Express to read the real client IP and protocol from proxy headers. It matters for rate limiting and secure cookies.

More frequently asked

Q49. How would you structure an Express + MongoDB project? ★ Most asked

```
server/
  config/db.js          -> database connection
  models/User.js        -> Mongoose schemas
  controllers/userController.js -> logic
  routes/userRoutes.js  -> URL to controller mapping
  middleware/auth.js    -> JWT check, error handler
  utils/                -> helpers
  server.js             -> app setup, middleware, listen
  .env
```

Keeping routes thin and logic in controllers (or services) makes the code easy to test and maintain.

Q50. What is the difference between `app.use()` and `app.get()`?

`app.use(path, fn)` runs for every HTTP method and matches any path that starts with the given path (used for middleware and mounting routers). `app.get(path, fn)` only handles GET requests on that exact path.

Q51. How do you handle cookies securely?

Set `httpOnly: true` (JS can't read it, protects against XSS), `secure: true` (HTTPS only), `sameSite: "strict"` or `"lax"` (protects against CSRF), and an expiry with `maxAge`.

Q52. What is the difference between Express 4 and Express 5?

Express 5 automatically forwards errors from async handlers/rejected promises to the error middleware, uses an updated path-matching syntax (wildcards must be named, optional segments use braces), removes some old deprecated methods, and requires a newer Node version.